Imprecise Computations

JANE W. S. LIU, WEI-KUAN SHIH, KWEI-JAY LIN, RICCARDO BETTATI, AND JEN-YAO CHUNG, MEMBER, IEEE

Invited Paper

The imprecise computation technique has been proposed as a way to handle transient overload and to enhance fault tolerance of real-time systems. In a system based on this technique, each time-critical task is designed in such a way that it can produce a usable, approximate result in time whenever a failure or overload prevents it from producing the desired, precise result. This paper describes ways to implement imprecise computations, models to characterize them and algorithms for scheduling them. An imprecise mechanism for the generation and use of approximate results can be integrated in a natural way with a traditional fault-tolerance mechanism. An architectural framework for this integration is described.

I. Introduction

In a (hard) real-time system, many tasks are time-critical. Here, by *task*, we mean a basic unit of work, such as a granule of computation, a unit of data transmission, a file transfer, etc. Time-critical tasks have timing constraints. The *timing constraint* of a task is typically given in terms of its *feasible interval*. This interval begins at the task's *ready time*; the task can begin execution at or after this time. The interval ends at the task's *deadline*; the task must complete and produce its result by its deadline. A task's failure in producing its result on time is a timing fault. A real-time system works correctly only when there is no timing faults. Unfortunately, timing faults invariably occur when a real-time system becomes overloaded, and overload conditions are unavoidable.

A way to avoid timing faults during transient overloads and to make a real-time system responsive and robust is to use the *imprecise computation technique* [1]–[6]. We call a system based on this technique an *imprecise system*. In an imprecise system, each time-critical task, or a set of tasks, contains an optional part. Under normal operating conditions, the optional part is completed, and the result of the task has the desired quality and is said to be *precise*. During overloads, however, the optional part, or a portion of it, can be skipped (that is, left unfinished) in order to conserve system resources. When its optional part is skipped, the task produces an *imprecise* (i.e., approximate) result. Examples of applications where one may prefer timely imprecise results, to late precise results are image processing and tracking. It is often better to have frames of fuzzy images or estimates of target locations in time than perfect images or accurate location data too late, or no image or data at all.

The imprecise computation technique can also be used in a natural way to enhance fault tolerance and graceful degradation. We consider here only hardware faults and failures, as well as transient software faults that disappear after the system software is reset to a fault-free state. Permanent software faults are not considered. To see how availability and fault tolerance can be increased in an imprecise system, we consider tracking and control systems for example. A transient fault may cause a tracking computation to terminate prematurely and produce an approximate result. No recovery action is needed if the result allows the system to maintain track of the targets. Similarly, as long as the approximate result produced by a control law computation is sufficiently accurate for the controlled system to remain stable, the fault that causes the computation to terminate prematurely can be tolerated. In time-critical, embedded systems, there is often no time to take any recovery action, and limited resources do not allow full replication of data storage and task execution. Providing usable, approximate results in the presence of failures is a way to increase the availability of data and services, to reduce the need for error-recovery operations, to simplify the recovery operations that must be done, and to reduce the costs in replication.

This paper gives an overview of key issues and recent results on algorithms for scheduling imprecise computations

Manuscript received May 31, 1993; revised July 13, 1993. This work was partially supported by the U.S. Navy under ONR Contracts NVY N0001489-J-1181 and NVY N0001489-J-1146.

J. W. S. Liu, W. K. Shih, and R. Battati are with the Department of Computer Science, University of Illinois at Urbana-Champaign, Urbana, IL 61801.

K. J. Lin was with the Department of Computer Science, University of Illinois, Urbana-Champaign, Urbana, IL 61801. He is now with the Department of Electrical and Computer Engineering, University of California, Irvine, CA 92664.

J. Y. Chung is with IBM T. J. Watson Research Center, Yorktown Heights, NY 10598.

IEEE Log Number 9214156.

0018–9219/94$04.00 © 1994 IEEE

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 07,2025 at 22:54:59 UTC from IEEE Xplore. Restrictions apply.

and imprecise system architectures for real-time fault tolerance. Following this introduction, Section II describes the different types of imprecise computations and the models used to characterize them. Section III provides an overview of the problems on scheduling imprecise computations and a brief survey of the known results. Section IV describes the key elements of an imprecise system architecture that integrates the imprecision and fault-tolerance mechanisms to support the use of imprecise computations. Section V is a summary and discusses future directions.

II. Imprecise Computations

To see the rationale of the imprecise computation technique, we note that the bad effects of timing faults are often tolerable as long as all the important tasks are completed in time. Therefore, rather than letting the operating system treat all tasks equally, the programmer may identify some tasks as *mandatory*, meaning that they must be completed in their feasible intervals, and some less important tasks as *optional*, meaning that these tasks can be skipped without causing intolerable timing faults. In this way, the operating system is allowed to skip the less important tasks during overloads so that important tasks can complete in time.

A. Implementation Methods

We often can skip a task whose sole purpose is to produce an output that is at least as precise as the corresponding input or an old output. We call such a task a *sieve* function. Typically, there is no benefit by completing a sieve in part. Therefore, we want to either execute such an optional task to completion before its deadline or skip it entirely. In this case, we say that the execution of the optional task satisfies the *0/1 constraint*. When tasks have the 0/1 constraint, the scheduler must decide before the execution of each task begins whether to schedule the task entirely or to skip it entirely. Some scheduling flexibility is thus lost.

A task, as well as the underlying computational algorithm, is said to be *monotone* if the quality of the intermediate result produced by it is nondecreasing as it executes longer [7]–[9]. We can logically decompose each monotone task into a mandatory task and an optional task. The result produced by a monotone task when the entire task completes is the precise result. If the task is terminated before it is completed, the imprecise result produced by it at the time of its termination is the best among all the intermediate results produced before its termination. This result is usable as long as its mandatory task is completed. Because the scheduler can terminate a task any time after it has produced a usable result, scheduling monotone tasks can be done on-line or nearly on-line, and the decision on which optional tasks and how much of each optional task to schedule can be made dynamically. Consequently, we have the maximum flexibility in scheduling when imprecise computations are monotone.

Using *multiple versions* of tasks [10]–[14] is another way to provide scheduling flexibility. A multiple-version task has at least two versions: the primary version and the alternate version(s). The primary version of each task produces a desired, precise result but has a longer processing time. An alternate version has a shorter processing time but produces an imprecise result. During a transient overload, when it is not possible to complete the primary version of every task by its deadline, the system may choose to schedule alternate versions of some tasks. For scheduling purpose, we can view the alternate version of each two-version task as a mandatory task and the primary version as a mandatory task followed by an optional task. The processing time of the mandatory task is equal to that of the alternate version, and the processing time of the optional task is equal to the difference between the processing times of the two versions. The optional task being scheduled (or skipped) entirely corresponds to the primary version (or the alternate version) being scheduled. Therefore, algorithms for scheduling tasks with the 0/1 constraints can be used to schedule two-version tasks.

B. The Basic Imprecise Computation Model

Most of the workload models used to characterize imprecise computations are based on the classical deterministic model of real-time applications, which characterizes an application system as a set $\boldsymbol{T}$ of tasks. The tasks may be dependent; data and control dependencies between tasks impose precedence constraints on the order of their execution. We use a partial-order relation $<$ over $\boldsymbol{T}$ to specify these dependencies. A task T_i is a *predecessor* of another task T_j (and T_j a *successor* of T_i), denoted as $T_i < T_j$, if T_j cannot begin execution until the execution of T_i completes. T_i is an *immediate predecessor* of T_j (and T_j is an *immediate successor* of T_i) if $T_i < T_j$ and there is no other task T_k such that $T_i < T_k < T_j$. Two tasks T_i and T_j are *independent* when neither $T_i < T_j$ nor $T_j < T_i$; they can be executed in any order.

In the basic imprecise-computation model, each task T_i in the system is defined by the following parameters, which are rational numbers.

1) *Ready time* r_i' is the time instant at which T_i becomes ready for execution.
2) *Deadline* d_i' is the time instant by which T_i must be completed.
3) *Processing time* τ_i is the time required to execute the entire task T_i to completion on a processor.
4) *Weight* w_i is a positive number and measures the relative importance of the task.

Each task T_i is decomposed logically into two tasks, the *mandatory* task M_i and the *optional* task O_i. The ready time and deadline of the tasks M_i and O_i are the same as that of T_i, and M_i is the immediate predecessor of O_i. Let m_i and o_i be the processing times of M_i and O_i, respectively. m_i and o_i are rational numbers, and $m_i + o_i = \tau_i$. The classical deterministic model is a special case of the imprecise computation model where $o_i = 0$ for all i. Similarly, the traditional soft-real-time workload model is also a special case in which all tasks are completely optional, that is, $m_i = 0$ for all i.

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 07,2025 at 22:54:59 UTC from IEEE Xplore. Restrictions apply.

We confine our attention here to case where the tasks in $\boldsymbol{T}$ are scheduled on a processor. A *schedule* is an assignment of the processor to the tasks in $\boldsymbol{T}$ in disjoint intervals of time. A task is said to be scheduled in a time interval if the processor is assigned to the task in the interval. The *amount of processor time* assigned to a task is the total length of the intervals in which the processor is assigned to the task. In any *valid schedule*, the processor is assigned to at most one task at any time, every task is scheduled after its ready time and all its predecessors have been completed, and the amount of processor time assigned to the mandatory task M_i of every task T_i is equal to m_i. The optional task O_i of a monotone task may be terminated at any time. Hence, the amount of processor time assigned to a monotone task is at least equal to m_i and at most equal to τ_i. The amount of processor time assigned to a task with a 0/1 constrained optional task is equal to either m_i or τ_i. When the amount of processor time σ_i assigned to an optional task O_i in a schedule is equal to o_i, we say that the task O_i (or the task T_i) is *precisely* scheduled. In a *precise schedule*, every task is precisely scheduled. Only precise schedules are valid schedules in the traditional sense. Otherwise, if σ_i is less than o_i, we say that the schedule is imprecise.

A task T_i is said to be completed in a schedule whenever its mandatory task is completed in the traditional sense. A valid schedule is a *feasible* schedule if in which every task is completed by its deadline. A set of tasks is said to be *schedulable* if it has at least one feasible schedule. It is schedulable by an algorithm if the algorithm always produces a feasible schedule.

Sometimes, rather than the ready times and deadlines of individual tasks in $\boldsymbol{T}$, only the end-to-end ready time and deadline of the entire set are given. The former is the time at which tasks without predecessors in the set can begin execution. The latter is the time by which tasks without successors in the set are required to complete. When tasks are scheduled on one processor, it is not necessary to assign immediate ready times and deadlines to tasks that have no given ready times and deadlines. It suffices for us to assume that all the tasks have the same ready time and deadline.

The given deadline of a task can be later than that of its successors, and, hence, can be inconsistent with the precedence constraints between tasks. Rather than working with the given deadlines, we use the modified deadlines that are consistent with the precedence constraints and are computed as follows. The modified deadline d_i of a task T_i that has no successors is equal to its given deadline d_i'. Let A_i be the set of all successors of T_i. Since T_i must be completed and terminated before its successors, the modified deadline d_i of T_i is $\min\left\{d_i', \min_{T_j \in A_i}\{d_j\}\right\}$. Similarly, we can compute in an analogous manner the modified ready time r_i that is consistent with the precedence constraints from the given ready times of the task T_i and all of its predecessors. It has been shown in [15] that a set $\boldsymbol{T}$ of tasks with the given ready times and deadlines is schedulable if and only if the set of tasks with the modified ready times and deadlines is schedulable. Working with the modified ready times and deadlines allows us to ignore the precedence constraints temporarily. If an algorithm finds an invalid schedule in which T_i is assigned a time interval later than some intervals assigned to T_j but $T_i < T_j$, a valid schedule can be constructed by exchanging the time intervals assigned to T_i and T_j to satisfy their precedence constraints. This exchange can always be done without violating their timing constraints. In our subsequent discussion, by ready times and deadlines, we mean modified ready times and deadlines and call the interval $[r_i, d_i]$ the feasible interval of the task T_i.

C. Error and Error Functions

When tasks are dependent on each other, the result produced by a task may be an input of its immediate successors. When this result is imprecise, the input is erroneous. Let the errors in the inputs of a task T_i be denoted by the vector $\boldsymbol{e}_i$. The error in the result produced by T_i whose optional task is assigned σ_i units of processor time is given by its *error function* $\varepsilon_i = E_i(\sigma_i, \boldsymbol{e}_i)$, which is a nonincreasing function of σ_i and a nondecreasing function of $\boldsymbol{e}_i$. A task may need to do some additional work to correct its input error, and a poorer input may slow down the rate at which its result converges to the precise one. Consequently, the processing times of both M_i and O_i may become larger as the input error $\boldsymbol{e}_i$ increases in magnitude; both $m_i(\boldsymbol{e}_i)$ and $o_i(\boldsymbol{e}_i)$ are nondecreasing functions of $\boldsymbol{e}_i$. When the magnitude of $\boldsymbol{e}_i$ becomes larger than some threshold, $o_i(\boldsymbol{e}_i)$ may become infinite, and the task T_i can never produce a precise result no matter how long it executes. The need to keep the error in an input below this threshold determines when a result produced by each predecessor task is acceptable and how large the processing time of the mandatory part of the predecessor task should be.

Almost all past studies on imprecise computation scheduling neglects the effect of input error. In other words, the error of T_i is equal to $\varepsilon_i = E_i(\sigma_i)$ where the error function $E_i(\sigma_i)$ is only a function of σ_i and is monotone nonincreasing in σ_i. $E_i(o_i)$ is zero.

While the error function typically decreases in discrete steps and is, therefore, discontinuous, we often can approximate it accurately by a continuous function [4]. Because mandatory tasks are always precisely scheduled, the value of the error in the range where the amount of assigned processor time is less than the mandatory processing time is unimportant. For the sake of convenience, we let this value be 1. The exact behavior of the error function depends on the nature of the underlying computational algorithm and it is often not known. A reasonable choice is to assume that the error of a task is equal to $o_i - \sigma_i$, the processing time of its skipped portion. This *linear error function* is commonly used in studies on scheduling imprecise computations. (Other types of error functions are convex and concave functions of $o_i - \sigma_i$.) For a given schedule, the *total error*

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 07,2025 at 22:54:59 UTC from IEEE Xplore. Restrictions apply.

of the task set $\boldsymbol{T}$ is

$$\varepsilon = \sum_{i=1}^{n} w_i \varepsilon_i.$$

Again, $w_i > 0$ are the weights of the tasks.

III. Imprecise-Computation Scheduling

In this section, we provide a brief overview of the recent results on imprecise-computation scheduling. Except for the case of periodic jobs discussed in Section III-B, the effects of input errors are neglected.

A. Scheduling Monotone Imprecise Computations to Minimize Total Error

All the imprecise-scheduling problems that have been solved to date are variations and extensions of the following one. We have a set of tasks $\boldsymbol{T} = \{T_1, T_2, \cdots, T_n\}$, where all tasks are monotone and have identical weights. The error of every task is equal to the processing time of the skipped optional portion. We want to find a feasible preemptive schedule of $\boldsymbol{T}$ on a processor that has the minimum total error among all feasible schedules. Such a schedule is said to be optimal. An *optimal scheduling algorithm* always finds an optimal schedule whenever the given set of tasks is schedulable.

We classify tasks as on-line and off-line. Tasks whose parameters are known to the scheduler before the processor starts to execute any task are *off-line tasks*, and tasks that arrive after the execution of some tasks has started are *on-line tasks*. The parameters of an on-line task become known when it arrives. A scheduling algorithm is off-line if it assumes that all the tasks to be scheduled are off-line. An algorithm is on-line when it can deal with on-line tasks.

1) Optimal Off-Line Algorithm: An optimal off-line algorithm for scheduling tasks with identical weights is *Algorithm F* described in [5], [6]. This algorithm makes use of a modified version of the classical, earliest-deadline-first algorithm [15], called the *ED Algorithm.* This is a preemptive, priority-driven algorithm which assigns priorities to tasks according to their deadlines; tasks with earlier deadlines have higher priorities. However, according to the ED Algorithm, every task is terminated at its deadline even if it is not completed at the time. In any ED schedule produced by this algorithm, every task is scheduled in its feasibility interval. When all tasks are entirely optional, an ED schedule is feasible by definition. Because such a schedule is priority-driven, the processor is never left idle when there are tasks ready to execute. A portion of an optional task is skipped only when it is necessary to do so. Therefore, the ED Algorithm is optimal when used to schedule optional tasks that have identical weights.

Algorithm F makes use of the fact that the ED Algorithm produces a schedule with the minimum total error. It first uses the ED Algorithm to find a schedule S_t of $\boldsymbol{T}$, treating all mandatory tasks as optional tasks. An optimal schedule of $\boldsymbol{T}$ is obtained if the resultant schedule has zero error. Otherwise, the ED Algorithm is used to test whether the task set $\boldsymbol{T}$ can be feasibly scheduled. The feasibility test is done by scheduling the mandatory set $\boldsymbol{M}$ alone using the ED Algorithm. The task set $\boldsymbol{T}$ can be feasibly scheduled if the resultant ED schedule S_m of $\boldsymbol{M}$ is precise. Because of the optimality of the classical, earliest-deadline-first algorithm [15], we can conclude that no feasible schedule of $\boldsymbol{T}$ exists if the schedule S_m is not precise. After $\boldsymbol{T}$ is found to be feasible, Algorithm F then transforms S_t into an optimal schedule that is feasible and minimizes the total error. This transformation process uses S_m as a template and adjusts the amounts of processor times assigned to the tasks so that a sufficient amount of processor time is assigned to every mandatory task for it to complete by its deadline, and no additional idle time is introduced during this transformation process. The complexity of Algorithm F is the same as that of the ED Algorithm, that is, $O(n \log n)$.

2) On-Line Algorithms: Algorithm F cannot handle on-line scheduling efficiently. It is well known that there can be no on-line algorithms that are optimal in the sense defined earlier. In practice, on-line tasks whose mandatory parts cannot be feasibly scheduled at their arrival times are not accepted by the system. These tasks are lost but are handled by some exception mechanism provided by the application. Consequently, the tasks presented to the scheduler satisfies the *feasible mandatory constraint.* A set of tasks to be scheduled satisfies this constraint if at the arrival time of every on-line task, its mandatory part, together with the yet-to-be-completed mandatory parts of tasks that have already arrived, can be scheduled to complete by its deadline.

When the feasible mandatory constraint is satisfied, there are on-line algorithms that are optimal in the sense that they produce feasible schedules with minimum total error [16]. Depending on when on-line tasks are ready and whether there are off-line tasks, we have the following four cases: 1) there are no off-line tasks, and every on-line task is ready for execution when it arrives; 2) there are off-line tasks, and every on-line task is ready for execution when it arrives; 3) there are no off-line tasks, and on-line tasks have arbitrary ready times; and 4) there are off-line tasks, and on-line tasks have arbitrary ready times.

The on-line algorithms for all four cases can be described by the pseudo-code in Fig. 1. They are similar in two ways. First, each algorithm maintains a data structure, called the *reservation list.* The reservation list contains a reservation for each of the tasks that has arrived but is not completed. Specifically, the *reservation* of each task T_i consisting of one or more reserved (time) intervals in its feasible interval $[r_i, d_i]$. When a reservation is made for a task T_i, its reserved intervals are inserted in the reservation list. The reserved intervals are found by reverse scheduling the mandatory part of the task, together with all yet-to-be-completed mandatory portions of all tasks, from the latest deadline to the current time in the following manner: Each task is scheduled at or before its deadline, and tasks are scheduled in the latest-ready-time-first order. An interval is reserved for a task if it is assigned in the reverse schedule to the task.

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 07,2025 at 22:54:59 UTC from IEEE Xplore. Restrictions apply.

```
0.  Generate the reservation list containing reservations of all off-line tasks if any.
    Put all ready tasks in the task queue ordered on the earliest-deadline-first basis.
1.  For as long as the task queue is not empty and no event occurs, execute the task at the
    head of the task queue.
2.  When an event occurs:
    2.1  Event_1, a task becomes ready:
         -- update the reservation of the current task;
         -- put the newly ready task in the task queue;
         -- goto step 1.
    2.2  Event_2, the mandatory part of the current task completes:
         -- cancel the reservation of the current task;
         -- goto step 1.
    2.3  Event_3, the optional part of the current task completes or its deadline is
         reached:
         -- terminate the task and remove the task from the task queue;
         -- goto step 1.
    2.4  Event_4, a new on-line task arrives:
         -- if the current task is mandatory, update the reservation of the current task;
         -- make reservation for this new task;
         -- if this task is ready, insert it into the task queue;
         -- goto step 1.
    2.5  Event_5, the beginning of a reserved interval is reached:
         -- if the interval is reserved for the current task,
            then goto step 1
            else update the reservation of the current task;
                 -- put the task assigned to the reserved interval at the head of the task
                    queue;
                 -- goto step 1.
```

Fig. 1. Description of on-line algorithms.

Initially, when a reservation is first made, the total length of all reserved intervals of T_i is equal to the processing time m_i of the mandatory task M_i. The on-line scheduling algorithms use the reservation list as a guide. They never schedule any optional task in a reserved interval. In this way, the algorithms ensure that a sufficient amount of time is assigned to every mandatory task for it to complete by its deadline. The reservation list is updated as the execution of each mandatory task M_i progresses to completion. At each update, the length of its reservation is reduced to the processing time of the to-be-completed portion, and the reservation is eventually deleted when M_i completes, canceling the reservation of M_i.

The second way in which the on-line algorithms are similar is that at any scheduling decision time, they schedule tasks on the earlist-deadline-first (EDF) basis. The scheduler maintains a prioritized task queue in which tasks are ordered on the EDF basis. The scheduler makes reservation for each task as it arrives, puts the task into the task queue when it becomes ready, and fetches the first task from the task queue for execution. For every task, the processor always executes the mandatory task first, and then executes the optional task. The optional task executes as long as there remains to be time prior to the beginning of the earliest reserved interval. The optional task is terminated when the beginning of a reserved interval is reached. When a task is terminated, it is removed from the task queue. In a reserved interval, the ready mandatory tasks are scheduled on the EDF basis.

The algorithms for the different cases differ in the data structures they use to maintain the reservation list and the operations to insert, update, and cancel reservations. The simpliest is case 1) where all tasks are on-line and ready upon arrival. In this case, there is no need to keep track of the identities of the tasks assigned to individual reserved intervals, only the instants when the intervals begin and end and the total amount of time reserved for each task. New reserved intervals can be added and existing reserved intervals can be shortened and deleted in constant time. The complexity of the on-line algorithm for this case, called the *NORA Algorithm*, is $O(n \log n)$. The algorithm for case 2), where there are off-line tasks with arbitrary ready times but on-line tasks are ready upon arrival, also has this complexity. Cases 3) and 4), where the ready times of all tasks are arbitrary, are more complex. We need to know where the reserved intervals of every task are. Making a new reservation or canceling an existing reservation requires $O(\log^2 n)$ time. The on-line algorithm for these cases, called the *ORA Algorithm*, has time complexity $O(n \log^2 n)$.

B. Scheduling Periodic Tasks to Minimize Average Error

A variation of the basic imprecise computation model is the imprecise-periodic-job model [4]. As in the well-known periodic-job model [17], the workload to be scheduled is a set $\boldsymbol{J}$ of n independent periodic jobs. Each job consists of a periodic sequence of identical tasks. The ready time and the deadline of the task in each period is the beginning and the end of the period, respectively. The *period* π_i of a job J_i in $\boldsymbol{J}$ is the length of the time interval between the ready times of two consecutive tasks in the job. The processing time τ_i of job J_i is the amount of processor time required to complete each task in J_i. Each task in J_i consists of a mandatory task, that has processing time m_i, and an optional task, that has processing time $\tau_i - m_i$ and is dependent on the mandatory task. In other words, each job J_i characterized by the tuple (π_i, τ_i) is decomposed into two periodic jobs: the mandatory job $M_i = (\pi_i, m_i)$ and the optional job $O_i = (\pi_i, \tau_i - m_i)$.

We divide applications in two types: error-noncumulative or error-cumulative. For any *error-noncumulative application*, tasks in different periods are independent and their inputs are precise. Only the average effect of errors in different periods is observable and relevant. Optional tasks are truly optional. An example of this type of application is image or video transmission. In contrast, for an *error-cumulative application*, tasks in different periods of each job are dependent. Because of input error, when the optional tasks in several consecutive periods are not completed, their errors accumulate. A simplifying assumption is that the error accumulated over these periods is reset to zero when the optional task in the subsequent period completes and produces a precise result. To keep the cumulative error below a tolerable threshold, an optional task in several consecutive periods must be completed and, hence, is no longer optional. Examples of this type of application include tracking and control.

Different performance metrics are appropriate for different types of applications. An appropriate performance metric for measuring the relative merits of schedules of error-noncumulative jobs and the performance of scheduling algorithms is the average error in the results produced in consecutive periods. The average error ε_i of J_i at any time is the average value of $E_i(\sigma_{ij})$ over the past π/π_i periods, where σ_{ij} is the amount of processor time assigned to the

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 07,2025 at 22:54:59 UTC from IEEE Xplore. Restrictions apply.

task in the jth period of J_i and π is the least common multiple of all the periods. The average error over all jobs in $\boldsymbol{J}$ is

$$\varepsilon = \sum_{i=1}^{n} w_i \varepsilon_i$$

where w_i is a nonnegative constant weight and

$$\sum_{i=1}^{n} w_i = 1.$$

It meaures the overall quality of results produced by all jobs.

There is a class of heuristic algorithms for scheduling error-noncumulative jobs on a processor to minimize the average error. All the algorithms in this class are preemptive and priority-driven, and all use the following strategy: optional tasks are executed only after all the ready mandatory tasks have completed, and all optional tasks are terminated at their deadlines. Specifically, given a job set $\boldsymbol{J}$, all the mandatory jobs have higher priorities than all the optional jobs. Moreover, mandatory jobs are scheduled precisely according to the rate-monotonic algorithm [17]. When the total fraction of time during which all mandatory jobs keep the processor busy is equal to or less than $n(2^{1/n} - 1)$, the set of all mandatory jobs can always be feasibly scheduled according to this algorithm. All the deadlines can be met regardless of how optional jobs are scheduled.

The algorithms in this class differ only in the ways in which they assign priorities to optional jobs. Some of the algorithms make priority assignments to optional tasks based on the behavior of error functions. Examples of these algorithms include the *least-utilization algorithm*, which statically assigns higher priorities to optional jobs with smaller weighted utilization factors $(\tau_i - m_k)/\pi_i w_i$, and the *least-attained-time algorithm*, which assigns the highest priority at any time to the optional task that has attained the least processor time at the time among all ready optional tasks. The former tends to perform well when the error functions of all tasks are linear, while the latter performs well when the error functions are convex. When the error functions are concave, optional tasks should be scheduled on the first-in-first-out basis.

Often, the exact behavior of the error functions is unknown, and different jobs have different types of error functions. An algorithm that ignores the error functions but attempts to minimize the average error by assigning higher priorities to more urgent optional tasks is more suitable. Examples of such algorithms are the *shortest-period algorithm*, which statically assigns priorities to optional jobs on the rate-monotonic basis, and the *earliest-deadline algorithm*, which assigns priorities dynamically to optional tasks on the earliest-deadline-first basis. It has been shown that the earliest-deadline algorithm is optimal among this class of algorithms in the sense that this algorithm can always find a precise schedule and achieve zero error whenever any algorithm in this class can precisely schedule the given set of tasks.

The primary objective of scheduling error-cumulative jobs is to ensure that all mandatory tasks are completed by their deadlines and an optional task in a given number of consecutive periods is completed by its deadline. This scheduling problem is NP-hard even in the simpliest case when all jobs have identical periods and error functions [4]. An approximate algorithm with bounded performance is known only for this simple case. Several effective heuristic algorithms for scheduling error-cumulative jobs that have arbitrary periods and error functions have recently been developed and evaluated [18]. These algorithms use the same strategy as the algorithms for scheduling error-noncumulative jobs but choose to consider some optional tasks mandatory and schedule them as if they were mandatory tasks. The algorithms differ from each other in the rules they use to decide when optional tasks are treated as mandatory tasks. Examples are the *first-period strategy* (or the last-period strategy), where the optional task in the first (or the last) period among a given number of consecutive periods is considered mandatory, the *random strategy*, where the optional task in every period has a constant probability of being mandatory, and the *random-increasing strategy*, where the probability of an optional task being scheduled as a mandatory task increases when the cumulative error increases. By adjusting the constant probability or rate of increase in probability, the random strategies ensure that a sufficient number of optional tasks complete in time.

All of the algorithms for scheduling imprecise periodic jobs described above have the advantage of the rate-monotonic algorithm: deadlines will be missed in a predictable manner during a transient overload. They also have the advantage of allowing the processor to be fully utilized like the classical, earliest-deadline-first algorithm. They are ideally suited when transient overloads occur frequently or when the actual processing times of tasks vary widely. In most cases, the average error remains tolerably small when the total utilization of $\boldsymbol{J}$ becomes larger than 1 and no classical algorithm can schedule the tasks satisfactorily. On the other hand, these algorithms are not optimal. They may not be able to find precise, feasible schedules of some job sets that are schedulable by the classical rate-monotonic or earliest-deadline-first algorithms.

C. Other Imprecise Computation Scheduling Problems

Depending on the applications, errors in imprecise results lead to different consequences. Consequently, there are many different imprecise computation scheduling problems where different constraints are considered and different performance criteria are optimalized. Most of these problems are extensions of the basic one discussed in Section III-A. Examples are problems of scheduling tasks with different weights and tasks with the 0/1 constraint, as well as scheduling to minimize the maximum error, the number of tardy tasks, and the average response time.

1) Scheduling Tasks with Different Weights: An optimal off-line algorithm for scheduling monotone tasks with different weights to minimize the total weighted error is

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 07,2025 at 22:54:59 UTC from IEEE Xplore. Restrictions apply.

the *LWF (largest-weight-first) Algorithm* [5]. This algorithm repeatedly uses Algorithm F as a primitive to find first for each task T_i in turn, starting from the one with the largest weight and in the order of nonincreasing weights, the maximum amounts of processor time ${\sigma_i}^o$ that can be assigned to its optional task O_i. A feasible schedule in which the amounts of processor time assigned to all optional tasks are ${\sigma_i}^o$ is an optimal schedule. The complexity of the LWF Algorithm is $O(n^2 \log n)$. Recently, an improved algorithm with complexity $O(kn)$ has been found [19], where k is the number of distinct weights.

2) Scheduling to Minimize the Maximum Normalized Error: When the error functions are convex, the total error of all tasks can be kept small by making the ratio of the amount of processor time σ_i assigned to each optional task to its processing time as equal to that of the other optional tasks as possible. For a given schedule of a set $\boldsymbol{T}$ of tasks with identical weights, the *maximum normalized error* of the task set is

$$\varepsilon = \max_i \{(o_i - \sigma_i)/o_i\}.$$

We want to minimize the maximum normalized error of all tasks. There are polynomial-time algorithms for finding an optimal schedule with the smallest maximum normalized error and their running times are $O(n^2)$ [20], [21].

3) Scheduling with the 0/1 Constraint: In a schedule that satisfies the 0/1 constraint, the amount of processor time assigned to every optional task is equal to either o_i or 0. The general problem of scheduling tasks to meet the 0/1 constraint and timing constraints, as well as to minimize the total error, is NP-complete when the optional tasks have arbitrary processing times. A reasonable strategy for scheduling tasks with identical weights is the *LPF (longest-processing-time-first)* strategy. This strategy tries to schedule the optional tasks with longer processing times first. Given a set $\boldsymbol{T}$ of n tasks, let E_l and E_o be the total error of a schedule produced using this strategy and of an optimal schedule, respectively. It has been shown that $E_l \leq 3E_o$ [22].

Another reasonable performance metric is the number of skipped tasks. (For example, when scheduling tasks with multiple versions, we want to minimize the number of tasks whose alternate versions are scheduled, that is, the number of skipped optional tasks.) A reasonable strategy for scheduling to minimize the number of skipped optional tasks is the *SPF (shortest-processing-time-first)* strategy. This strategy tries to schedule the optional tasks with shorter processing times first. Given a set $\boldsymbol{T}$ of n tasks, let N_s and N_o be the numbers of optional tasks skipped in a schedule produced using this strategy and in an optimal schedule, respectively. It has been shown that $N_s \leq 2N_o$ [22].

When their optional tasks have identical processing times and weights, tasks with the 0/1 constraint can be optimally scheduled in $O(n \log n)$ time or $O(n^2)$ time depending on whether the tasks have identical or different ready times. These algorithms can be found in [23].

4) Scheduling to Minimize Number of Tardy Tasks: Often, the value of the total error of the tasks is not important, as long as it is lower than a certain acceptable limit. We may then want to minimize the number of tasks that are tardy (that is, they complete and terminate after their deadlines) for a given maximum, tolerable total error. The complexity of this problem has been studied by Leung *et al.* [19]. They presented a pseudo-polynomial-time algorithm and a fast heuristic algorithm for preemptive scheduling on a uniprocessor of tasks whose feasible intervals include each other. In the worst case, the number of tardy tasks in a schedule found by the heuristic algorithm is approximately equal to three times that in an optimal schedule.

5) Scheduling to Minimize Average Response Time: Given a schedule S, the finishing time $f(T_i, S)$ of a task T_i is the time instant at which the task is terminated or completed. The *mean flow time* of the tasks according to this schedule is equal to

$$F = \sum_{i=1}^{n} f(T_i, S)/n.$$

The mean flow time of the tasks measures the average response time, the average amount of time a task waits until it completes. The goal of scheduling is to minimize the mean flow time subject to the constraint that the total error is less than an acceptable value. Unfortunately, all but the simplest special cases of this scheduling problem are NP-hard [24]. This problem can also be formulated queuing theoretically [12], [14], [25]. For example, a queuing model of an imprecise multiprocessor system where imprecise computations are implemented by providing two versions of each task is described in [12], [25]. A task is serviced at the *full level* (or *reduced level*) when the primary version (or the alternate version) of the task is scheduled and executed. When the system load is light and the average response time is small, every task is serviced at the full level. When the system load becomes heavy, the overall response time of the system is reduced by servicing some tasks in the system at the reduced level. Such a scheduling scheme is called a *two-level scheduling discipline*. The design parameters of such a system are the ratio of the average reduced-level processing time and full-level processing time of tasks and the threshold queue length at and above which the system load is considered heavy. Performance data supporting the choices of these parameters to tradeoff between average response time and result quality can be found in [14], [25].

6) Scheduling on Multiprocessor Systems to Minimize Total Error: The problem of scheduling independent tasks on identical processors to minimize total error can be formulated as a network flow problem [26]. Optimal algorithms based on this formulation have time complexity $O(n^2 \log^2 n)$ when tasks have identical weights and $O(n^2 \log^3 n)$ when the tasks have different weights.

A task is said to be parallelizable if it can be executed in parallel on a number of processors in order to complete in less time. The degree of concurrency of any task in an interval refers to the number of processors on which the task executes in parallel in the interval. The problem of finding schedules of parallelizable tasks to minimize

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 07,2025 at 22:54:59 UTC from IEEE Xplore. Restrictions apply.

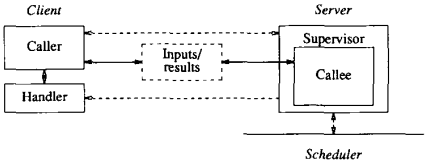


Fig. 2. A process structure.

the total error can be formulated as a linear programming problem. An algorithm for scheduling tasks whose degrees of concurrency are allowed to vary during execution can be found in [5].

IV. An Imprecise System Architecture

This section describes the key elements of an architectural framework that allows the integration of an *imprecision mechanism* for the generation and use of imprecise results with a traditional checkpointing and recovery mechanism for the enhancement of fault tolerance and availability. For the sake of concreteness, we assume that all imprecise computations are implemented using the milestone method. We will discuss briefly how multiple-versioned tasks can be used. Again, we consider here only transient faults. The mechanisms described here can be extended in a straightforward fashion to deal with permanent faults when there are redundant resources to allow hardware and software replications.

A. Result Saving, Application-Directed Checkpointing

In an imprecise system, the intermediate results produced by each task must be recorded while the task progresses towards completion. Figure 2 shows a process structure that can support the *result saving process*. It is a simple variation of the well-known client–server model. There is a server type for each service produced by an imprecise system. Each (instance of a) server (type) consists of a callee and a supervisor. The *callee* is either a procedure that has been initiated and made ready for activation or a task that may be separated from the client. Like the client, it is a part of the application written by the user. The input parameters provided by the client and the results produced by the callee are in a space that is accessible by both of them. The supervisor is a part of the underlying system. It is created at configuration time or at the instantiation time of the server. It may be dedicated to the server or shared by many servers. These details are unimportant for the purpose of our discussion here.

As in traditional systems, the callee executes whenever it is called by the client. The client calls it by sending an invocation request to its supervisor. When the supervisor grants the invocation request, it activates the callee. When the time allocated for the callee's execution expires, the supervisor terminates the callee.

The intermediate result variables and the accuracy indicator variables are defined by the programmer. During its execution, the callee records the values of these variables at instants that are also specified by the programmer, in the space that is accessible by the client. Each set of recorded values can be viewed as an *application-directed checkpoint*. The additional processing time required to record application-directed checkpoints is an unavoidable overhead. We can account for this overhead by including it in the processing time of the callee.

The client has the final, precise result when the callee terminates normally upon completion. If the system terminates it prematurely before it completes, the client has the best imprecise result produced by it before it terminates. The supervisor triggers a handler in the client whenever the callee terminates. Based on the latest recorded values of accuracy indicators, the handler can decide whether the imprecise result is acceptable. Again, the part of the callee that must be completed to produce an acceptable result is mandatory.

B. Periodic, System-Directed Checkpoint and Recovery

In addition to aiding the result-saving process that produces application-directed checkpoints, the supervisor checkpoints the callee and carries out sanity check on a periodic basis in order to enhance fault tolerance. When a failure is detected before the callee's mandatory portion completes, recovery is necessary. The supervisor restores the callee to its state at the time of the last checkpoint and resumes its execution from the state. We call such an action *system-directed recovery action.*

Because the supervisor and the callee do not communicate during callee's execution, a fault occurring during the callee's execution does not affect the supervisor. We further assume that the effect of any fault occurring during the callee's execution is confined to the callee. Moreover, sanity checks never fail. Each failure is detected by the next sanity check after the occurrence of the failure. These simplifying assumptions allow us to focus on the interaction between the imprecision and fault-tolerance mechanisms with sufficient generality.

Suppose that the supervisor checkpoints the callee every l units of time. At each checkpoint time, it executes for c units of time in order to perform a sanity check and generate a checkpoint. Let ρ denote the length of time the supervisor executes in order to roll back the callee. A server can tolerate k faults only when it is allocated a sufficient amount of processor time so that the mandatory part of the callee can complete before its deadline even when it must be rolled back k times. In other words, the scheduler must allocate to the mandatory part $m' = m + \lfloor m/l \rfloor c + k(l + c + \rho)$ units of processor time when its processing time is m.

The on-line algorithms described in Section III-A can be easily modified and used to schedule periodically checkpointed imprecise servers. Rather than reserving m units of processor time for each callee, which is referred to as a task in Fig. 1, the scheduler reserves m' units initially if the callee is required to tolerate k faults. When updating the reservation of the callee after y units of its mandatory part has completed and f failures have been detected and recovered from, the scheduler reduces the total length of its

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 07,2025 at 22:54:59 UTC from IEEE Xplore. Restrictions apply.

reservation to $(m-y)+\lfloor(m-y)/l\rfloor c+(k-f)(l+c+\rho)$. When a new invocation is granted, an Event_4 (arrival of a new task) occurs. Rather than updating only the reservation of the current task, the scheduler updates the reservation of all tasks whose mandatory parts remain incomplete. When all servers are ready to execute upon invocation (that is, all tasks are ready upon arrival), the simpler NORA Algorithm is suitable. This modification does not increase its complexity. When the execution of some servers may be delayed for arbitrary amounts of time upon invocation (that is, tasks have arbitrary ready times), we must use the more complicated ORA Algorithm. The modification increases its complexity to $O(n^2)$.

C. Imprecise Service Negotiation

Typically, only the client knows how accurate a result must be for it to be acceptable and how many faults each server invoked by it must tolerate. On the other hand, only the scheduler has the information on the demand and the availability of all resources. Both kinds of information are needed to support the decision on what quality of service to provide to each client and which tasks to terminate when an imprecise system recovers from a transient overload. Therefore, this decision should be made cooperatively between the application and the scheduler. In particular, to support this decision, the information regarding the result accuracy required by the client, the minimal level(s) of accuracy achievable by the server, and the required number of tolerated faults must be exchanged between the client, the server, and the scheduler at invocation time. The rules governing this interaction are given by an *imprecise service negotiation protocol.*

Figure 3 shows the interactions between the client, the server, and the scheduler during the invocation of an imprecise service. The information on the required level of accuracy, the deadline of the callee and the number of tolerated faults are passed by the client to the supervisor in its invocation request. The information about the callee maintained by the supervisor include the estimated error as a function of processsing time. Based on this information, the supervisor determines the estimated mandatory processing time of the callee and extends this time by the amount required for it to checkpoint the callee periodically and for the callee to roll back the requested number of times. It then sends this extended mandatory processing time, along with other attributes such as the estimated processing time of the optional portion and the resource requirements of the task, to the supervisor in its request to schedule the callee.

If sufficient processor time and system resources are available, the scheduler accepts the request and reserves a sufficient amount of time to ensure the completion of the callee by its deadline. It informs the supervisor, and the supervisor in turn informs the client, that the callee is scheduled and the invocation request is granted. The callee is placed on the task queue when it is ready and executes when it is at the head of the queue. When the callee terminates, the supervisor informs the client of its termination and that the result it produced is available.

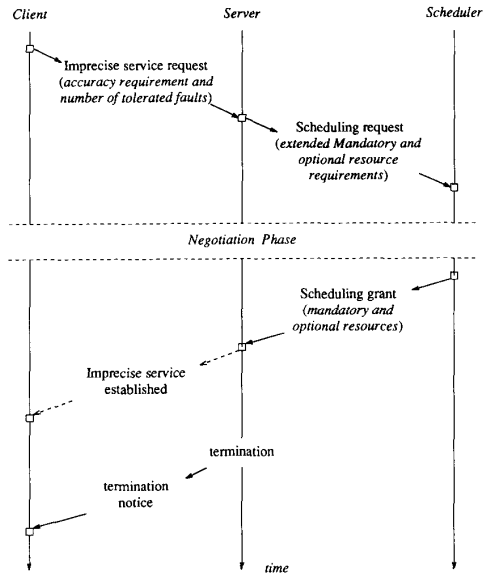


Fig. 3. Imprecise service establishment.

When it is not possible for the scheduler to allocate to the callee the requested amount of processor time, it informs the supervisor the available amount, rather than simply denying the request. The supervisor and the client may enter an optional phase where the quality of the requested service is negotiated. Sometimes the client can accept a larger error than it has requested. It may also be able to extend the deadline of the callee. This information can be included in its invocation request or passed to the supervisor during the negotiation phase. If the client can accept a result that can be produced with the available processor time, the callee is scheduled and invocation request is granted. Otherwise, the request is denied.

How the client recovers when its invocation request is denied is application-dependent. It may send an invocation request at a later time, for example. When alternate versions of the server are available, it may try to invoke an alternate server. The appropriate action is chosen and defined by the programmer.

D. Application-Directed Recovery

We call a recovery action initiated and controlled by the client an *application-directed recovery action.* Application-directed recovery is application-domain-specific. Whether some recovery action is necessary and what recovery action should be taken are likely to be dependent on the semantics of data, operations, and applications. The supervisor does not roll back the callee when it detects a failure during the execution of the optional part. It simply terminates the callee and notifies the client. Upon notification, the client may want to take some recovery action to improve the imprecise result returned by the callee.

Sometimes, the result returned by a server can be unacceptable even though the mandatory part of the callee

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 07,2025 at 22:54:59 UTC from IEEE Xplore. Restrictions apply.

Table 1 Imprecision Management Policies

Types of Application Domain	Decision Support Information or Recovery Actions		
	Type NC	Type LC	Type EC
Acceptance of x_i is based on	x_i	x_i and x_{i-1}	x_i,$x_{i-1}x_{i-2}, \cdots$
Recovery action depends on	x_i	x_i and x_{i-1}	x_i, x_{i-1}, $x_{i-2}, \cdots$
Alternative recovery actions:	refine x_i	refine x_i, or precompute x_{i+1}	refine x_i, or precompute x_{i+1}, or recompute x_{i-1}, $x_{i-2}, \cdots$

has been completed. This can occur for many reasons. In practice, it is often impossible to bound the time required to produce an acceptable result. As an example, we consider a task that iteratively finds a root of a polynomial. How long the task must execute to reduce the error to an acceptable level depends not only on the coefficients of the polynomial but also on the region where the root is expected to be found. The supervisor may underestimate the processing time of the mandatory part. Furthermore, the required accuracy of the result may depend on the value of the result; the client may find the result not sufficiently accurate after it examines the result.

We call a policy that defines the rules governing when and what application-directed recovery is done an *imprecision management policy* and the mechanism that enforces a set of such rules an *imprecise management mechanism*. To support a wide range of embedded, real-time applications, an imprecise system must provide three types of imprecision management mechanisms for three types of applications: (error) noncumulative (NC), limitedly cumulative (LC), and (error) cumulative (EC). The policies governing these mechanisms are summarized in Table 1. We let x_i denote the result produced by server T_i, for $i = \cdots i-1, i, i+1, \cdots$, invoked by the client in a sequence of time instants $\cdots t_{i-2} < t_{i-1} < t_i < t_{i+1}, \cdots$. When each result is available, the client must decide whether to accept the result as it is or try to improve it in some way. Table 1 gives the alternative recovery actions that are appropriate for different application types. More detailed descriptions of the mechanisms can be found in [6].

V. Summary

We have described here the imprecise computation technique and its applications. This technique allows us to transform the difficult problem of scheduling to meet deadlines to the easier problem of scheduling to trade off the result quality for overall system responsiveness and robustness. There are several models of imprecise computations that explicitly quantify the costs and benefits in this tradeoff. A scheduler in an imprecise system must balance the benefit in enhanced system response and availability against the cost in reduced result quality. For different types of applications, the cost and benefit are more appropriately measured by different criteria. We have presented an overview of recent results on scheduling tasks in imprecise systems to achieve optimal tradeoffs according to the different criteria.

We have also described an architectural framework of imprecise, fault-tolerant systems. The essential elements of an imprecise system are imprecise service establishment and result-saving protocols for the generation and use of imprecise results, imprecision management policies to ensure correct usage of imprecise results, and application-directed recovery making use of imprecise results. This imprecision mechanism can be integrated with a fault-tolerance mechanism that carries out system-directed checkpoint and recovery. A way to do this integration has been described.

Clearly, the result saving process must be efficient so that it will not degrade the normal system performance substantially. Similarly, the overhead incurred in the imprecise service establishment must be kept low. We are evaluating several alternative ways to implement these protocols to determine how best to meet these performance objectives. To make the imprecise computation technique readily applicable, domain-specific imprecision management policies for a wide range of real-time applications need to be fully developed. Such a policy is possible only in application domains where there are monotone computational algorithms that support meaningful and usable semantics for interpreting imprecise results as approximations of precise results and when there are quality metrics for comparing the approximations. Such algorithms remain to be developed for many important application domains, including database updates and control.

Ideally, one would like to derive the rules defining the imprecise management policies in an imprecise system from the requirements of the system. An imprecise system should continue to provide its services at some acceptable, although degraded, level when acceptable imprecise results are used in place of the desired results. These degraded levels are defined by a series of specifications of the system requirements. For example, in a simplistic way, an imprecise system can be viewed as having a set of minimal requirements and a set of enhanced requirements. In a correct implementation of the system, the minimal requirements are met whenever all the mandatory tasks are completed and all the imprecise results are acceptable. The enhanced requirements are met when all the tasks are executed to completion. How the criteria for accepting imprecise results can be derived from these requirement specifications remains to be investigated.

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 07,2025 at 22:54:59 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] J. W. S. Liu, K. J. Lin, and C. L. Liu, "A position paper for the IEEE 1987 Workshop on Real-Time Operating Systems," Cambridge, MA, May 1987.

[2] K. J. Lin, S. Natarajan, and J. W. S. Liu, "Imprecise results: utilizing partial computations in real-time systems," in *Proc. IEEE 8th Real-Time Systems Symp.* (San Jose, CA), Dec. 1987.

[3] ——, "Scheduling real-time, periodic jobs using imprecise results," in *Proc. 8th Real-Time Systems Symp.* (San Jose, CA), Dec. 1987, pp. 252–260.

[4] J. Y. Chung, J. W. S. Liu, and K. J. Lin, "Scheduling periodic jobs that allow imprecise results," *IEEE Trans. Comput.*, vol. 39, no. 9, pp. 1156–1174, Sept. 1990. Also J.Y. Chung, Ph.D. dissertation, Dep. of Computer Science, Univ. of Illinois, Urbana, May 1989.

[5] J. W. S. Liu, K. J. Lin, W. K. Shih, A. C. Yu, J. Y. Chung, and W. Zhao, "Algorithms for scheduling imprecise computations," *IEEE Comput.*, pp. 58–68, May 1991.

[6] J. W. S. Liu, K. J. Lin, C. L. Liu, and W. K. Shih, "Imprecise computations: a means to provide scheduling flexibility and enhance dependability," in *Readings on Real-Time Systems*, C. M. Krishna and Y. H. Lee, Eds. New York: IEEE PRESS, 1992.

[7] S. Vrbsky and J. W. S. Liu, "An object-oriented query processor that returns monotonically improving answers," in *Proc. 7th IEEE Conf. on Data Engineering* (Kobe, Japan), Apr. 1991, pp. 472–481.

[8] P. Buneman, S. Davidson, and A. Watters, "A semantics for complex objects and approximate queries," in *Proc.7th Symp. on the Principles of Database Systems*, Mar. 1988, pp. 305–314.

[9] G. K. Wallace, "Overview of the JPEG (ISO/CCITT) still image compression standard," in *Visual Communication and Image Processing, '89* (SPIE, Philadelphia, PA), Nov. 1989.

[10] A. L. Liestman and R. H. Campbell, "A fault-tolerant scheduling problem," *IEEE Trans. Software Eng.*, vol. SE-12, no. 10, pp. 1089–1095, Oct. 1986.

[11] P. Gopinath and R. Gupta, "Applying compiler techniques to scheduling in real-time systems," in *Proc. 11th IEEE Real-Time Systems Symp.* (Orlando, FL), Dec. 1990.

[12] B. Kim and D. Towsley, "Dynamic flow control protocols for packet-switching multiplexers serving real-time multipacket messages," *IEEE Trans. Commun.*, vol. COM-34, no. 4, Apr. 1986.

[13] Y. Yemini, "A bang-bang principle for real-time transport protocols," in *Proc. SIGCOMM '83 Symp. Commun. Architect. Protocols*, May 1983, pp. 262–268.

[14] W. Zhao and E. K. P. Chong, "Performance evaluation of scheduling algorithms for dynamic imprecise soft real-time computer systems," *Australian Computer Sci. Commun.*, vol. 11, no. 1, pp 329–340, 1989.

[15] E. L. Lawler and J. M. Moore, "A functional equation and its application to resource allocation and scheduling problem," *Manag. Sci.*, vol. 16, pp. 77–84, 1969.

[16] W. K. Shih and J. W. S. Liu, "On-line scheduling of imprecise computations to minimize total error," in *Proc. 13th IEEE Real-Time Systems Symp.* (Phoenix, AZ), Dec. 1992, pp. 280–289.

[17] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard real-time environment," *J. Assoc. Comput. Mach.*, vol. 20, pp. 46–61, 1973.

[18] I. Cheong, "Heuristic algorithms for scheduling error-cumulative, periodic jobs," Ph.D. dissertation, Dep. of Computer Science, Univ. of Illinois, Jan. 1993.

[19] K. I. J Ho, V. K. M. Yu, and W. D. Wei, "Minimizing the weighted number of tardy tasks units," to appear in *Discrete Appl. Math.*.

J. Y-T. Leung and C. S. Wong, "Minimizing the number of late tasks with error constraints," in *Proc. 11th Real-Time Systems Symp.* (Orlando, FL), Dec. 1990.

[20] W. K. Shih and J. W. S. Liu, "Minimization of the maximum error of imprecise computations," to be published in *IEEE Trans. Comput.*

[21] K. I. J. Ho, J. Y. T. Leung and W. D. Wei, "Minimizing maximum weighted error of imprecise computation tasks," Tech. Rep., Dep. of Computer Science and Engineering, Univ. of Nebraska, 1992.

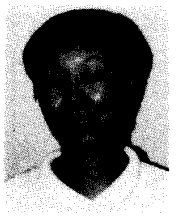
[22] K. I. J. Ho, J. Y. T. Leung, and W. D. Wei, "Scheduling imprecise computation tasks with 0/1 constraints," Tech. Rep., Dep. of Computer Science and Engineering, Univ. of Nebraska, 1992.

[23] W. K. Shih, J. W. S. Liu, and J. Y. Chung, "Algorithms for scheduling tasks to minimize total error," *SIAM J. Comput.*, vol. 20, no. 3, pp. 537–552, June 1991.

[24] J. Y-T. Leung, T. W. Tam, C. S. Wong, and G. H. Wong, "Minimizing mean flow time with error constraints," in *Proc. 10th IEEE Real-Time Systems Symp.*, Dec. 1989.

[25] W. Zhao, S. Vrbsky, and J. W. S. Liu, "An analytical model for multi-server imprecise systems," in *Proc. 5th Int. Conf. on Parallel and Distributed Computing and Systems* (Pittsburgh, PA), Sept. 1992.

[26] J. Y. Chung, W. K. Shih, J. W. S. Liu, and D. W. Gillies, "Scheduling imprecise computations to minimize total error," *Microprocess. Microprogr.*, vol. 27, pp. 767–774, 1989.



Jane W. S. Liu received the B.Sci. degree in electrical engineering in 1959 from Cleveland State University, Cleveland, OH, the M.Sc. and E.E. degrees in 1966, and the D. Sc. degree in 1968 from the Massachusetts Institute of Technology, Cambridge.

She is currently a Professor of Computer Science and Electrical and Computer Engineering at the University of Illinois at Urbana-Champaign. Her research interests are in the areas of databases, real-time systems, distributed systems, and computer networks.

Dr. Liu is a member of ACM and the IEEE Computer Society.



Wei-Kuan Shih received the B.S. and M.S. degrees in computer science from the National Taiwan University, and the Ph.D. degree in computer science from the University of Illinois at Urbana-Champaign.

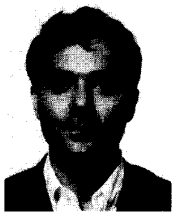
From 1986 to 1988, he was with the Institute of Information Science, Academia Sinica, Taiwan. He is an Associate Professor in the Department of Computer Science at the National Tsing Hua University, Taiwan. His research interests include real-time systems, scheduling theory, and VLSI design automation.



Kwei-Jay Lin received the B.S. degree in electrical engineering from the National Taiwan University, and the M.S. and Ph.D. degrees in computer science from the University of Maryland, College Park.

He was an Associacte Professor in the Department of Computer Science at the University of Illinois at Urbana-Champaign before moving to to the Department of Electrical and Computer Engineering at the University of California, Irvine, where he is now an Associate Professor. His research interests include real-time systems, scheduling theory, distributed systems, and fault-tolerant systems.

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 07,2025 at 22:54:59 UTC from IEEE Xplore. Restrictions apply.



Riccardo Bettati received the diploma in computer science from the Swiss Federal Institute of Technology (ETH), Zurich, Switzerland, in 1988. He is a Ph.D. candidate in the Department of Computer Science at the University of Illinois at Urbana-Champaign.

His research interests are in distributed real-time systems and real-time communication.

Mr. Bettati is a member of ACM and the Swiss Association of Computer Scientists.



Jen-Yao Chung (Member, IEEE) received the B.S. degree in computer science and information engineering from the National Taiwan University, Taipei, Taiwan, in 1982, and the M.S. and Ph.D. degrees in computer science from the University of Illinois at Urbana-Champaign, in 1987 and 1989, respectively.

Since June 1989, he has been with IBM Thomas J. Watson Research Center, Yorktown Heights, NY, as a research staff member, where he is currently working on parallel query processing and multisystem performance management. His research interests include parallel query procesing, job scheduling, and load balancing in real-time system, object-oriented programming environment, and operating system design.

Dr. Chung is a member of ACM, Tau Beta Pi, Sigma Xi, and Phi Kappa Phi.

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 07,2025 at 22:54:59 UTC from IEEE Xplore. Restrictions apply.